This time, once you've loaded the value of `head`, you must first increase the count of external references to the `head` node to indicate that you're referencing it and to ensure that it's safe to dereference it. If you dereference the pointer *before* increasing the reference count, another thread could free the node before you access it, thus leaving you with a dangling pointer. *This is the primary reason for using the split reference count:* by incrementing the external reference count, you ensure that the pointer remains valid for the duration of your access. The increment is done with a `compare_exchange_strong()` loop ❶ that compares and sets the whole structure to ensure that the pointer hasn't been changed by another thread in the meantime.

Once the count has been increased, you can safely dereference the `ptr` field of the value loaded from `head` in order to access the pointed-to node ❷. If the pointer is a null pointer, you're at the end of the list: no more entries. If the pointer isn't a null pointer, you can try to remove the node by a `compare_exchange_strong()` call on `head` ❸.

If the `compare_exchange_strong()` succeeds, you've taken ownership of the node and can swap out the `data` in preparation for returning it ❹. This ensures that the data isn't kept alive just because other threads accessing the stack happen to still have pointers to its node. Then you can add the external count to the internal count on the node with an atomic `fetch_add` ❻. If the reference count is now zero, the *previous* value (which is what `fetch_add` returns) was the negative of what you just added, in which case you can delete the node. It's important to note that the value you add is actually *two less* than the external count ❺; you've removed the node from the list, so you drop one off the count for that, and you're no longer accessing the node from this thread, so you drop another off the count for that. Whether or not you deleted the node, you've finished, so you can return the data ❼.

If the compare/exchange ❸*fails*, another thread removed your node before you did, or another thread added a new node to the stack. Either way, you need to start again with the fresh value of `head` returned by the compare/exchange call. But first you must decrease the reference count on